

String Mastery

6 patterns. ~35 problems. Pattern-first FAANG-prep cheatsheet.

[@rathoreadiitya](#) - Instagram

v1.0 · Updated 2026-05-22

Why this sheet exists

String problems are the warm-up question in 80% of FAANG screens. Easy on the surface, brutal on the edge cases. Candidates fumble them because they treat strings as a different topic from arrays - they're not. Strings are arrays of characters with a smaller alphabet, immutability, and a handful of specific patterns.

This sheet collapses string into 6 patterns: two-pointer, sliding window, frequency / anagram, palindrome, pattern match (KMP / Rabin-Karp), and string parsing (atoi family). Each pattern has a mental model, code walkthrough, traps, warm-up, and 3-4 LC problems in the exact order to solve them. Own these 6 and you cover roughly 95% of string problems FAANG asks.

How to use: Crash course → 5 min. Decision tree → internalise it. Patterns 1-6 in order - each has warm-up, walkthrough(s), traps, and curated practice. Cross-pattern traps → read once before any interview. Revision drill → 20 minutes the night before.

Inside this sheet

1. Crash course (5 min)
2. Pattern decision tree (problem → pattern in 10 seconds)
3. Pattern 1 - Two-pointer on a string
4. Pattern 2 - Sliding window on a string
5. Pattern 3 - Frequency counting / anagrams
6. Pattern 4 - Palindromes
7. Pattern 5 - Pattern matching (strStr / KMP / Rabin-Karp)
8. Pattern 6 - String parsing (atoi family)
9. Cross-pattern traps (read once before any interview)
10. Pre-interview revision drill (20 min)
11. 7-day study plan

Crash course (5 min)

Every string problem reduces to one of these six questions:

12. Move pointers from each end (reverse, compare, palindrome)? → Two-pointer (Pattern 1).
13. Find the best substring under a constraint (longest, shortest, count)? → Sliding window (Pattern 2).
14. Group, compare, or validate based on character frequency? → Counter (Pattern 3).
15. Anything about palindromes (substring, count)? → Expand around centre (Pattern 4).

16. Find pattern P in text T? → strStr / KMP (Pattern 5).
17. Convert / validate a formatted string (atoi, version, IP)? → Explicit state-machine parsing (Pattern 6).

Plus three Python truths that decide your time complexity:

- Strings are IMMUTABLE - concatenation in a loop is $O(N^2)$. Build a list, join at the end.
- Slicing is $O(K)$ - not free. Inside a loop, slices kill you.
- `.find()` and ``in`` are $O(N+M)$ average - the standard library is usually faster than your hand-rolled scan.

Big insight you keep missing: “string problem” doesn't mean a different family of algorithms. It means “array problem with the alphabet locked to chars and a handful of immutability gotchas.” Map every string problem to one of the 6 patterns above and the rest is execution.

Pattern decision tree (problem → pattern in 10 seconds)

What the problem says	Pattern
“Reverse” / “is palindrome” / “compare two strings ignoring X”	1 - Two-pointer
“Longest / shortest substring with property X”	2 - Sliding window
“Anagram” / “group anagrams” / “first unique char”	3 - Frequency counting
“Longest palindromic substring / count palindromic substrings”	4 - Palindromes
“Find needle in haystack” / “strStr” / “repeated substring”	5 - Pattern matching
“String to integer” / “version compare” / “valid IP” / “decode”	6 - String parsing
“Character replacement / edits / transforms” - sliding window OR DP	2 (or DP sheet)
“decode string” / “basic calculator” - stack-based parsing	6 (or Stack sheet)

The 6 patterns

Pattern 1 — Two-pointers on a string - reverse, compare, normalise

One-liner: Use two indices walking from opposite ends (or same end) to reverse, validate, or compare without an extra buffer. $O(N)$ time, $O(1)$ extra space.

Mental model

Mental model: Strings in Python are immutable, so true in-place reverse needs a list. But the index pattern is identical to two-pointer on arrays: $l = 0, r = n-1$, while $l < r$: do something; $l += 1; r -= 1$. The trick is identifying which character to SKIP (non-alnum, lowercase compare, sentinel).

Invariant: After each step, the substring between the original ends and the current pointers has been processed; everything inside $[l, r]$ is yet to come.

When to use same-end pointers (slow / fast): In-place compaction problems - “remove duplicates,” “write next non-X here.” Slow tracks the WRITE position, fast scans for the next valid character.

Spot signals

- “Reverse the string / words / vowels”
- “Is the string a palindrome” after stripping junk
- “Compare two version strings” / “backspace string compare” (skip pattern)
- “Remove all X from string” / “compact in place”
- Any problem where you'd be tempted to use string slicing repeatedly - indices win.

Walkthrough - LC 125 Valid Palindrome

Two pointers from each end. Skip non-alphanumeric on either side. Compare lowercased.

Python

```
def isPalindrome(s):
    l, r = 0, len(s) - 1
    while l < r:
        while l < r and not s[l].isalnum(): l += 1
        while l < r and not s[r].isalnum(): r -= 1
        if s[l].lower() != s[r].lower(): return False
        l += 1; r -= 1
    return True
```

Walkthrough - LC 151 Reverse Words in a String

Two-step: split-and-reverse is the easy Python win. Manually: reverse the whole string, then reverse each word in place, then collapse spaces. The 3-step trick is the bar-raiser version.

Python

```
def reverseWords(s):
    return ' '.join(reversed(s.split()))
```

Common traps

- Forgetting `l < r` in the inner skip loops. If you only check `l < r` outside, the skips can fly past `r` and crash on `s[len(s)]`.
- Using string slicing inside the loop (`s = s[1:]`). Each slice is $O(N)$ and turns the algorithm $O(N^2)$. Use indices.
- Comparing without lowercasing when the problem says “ignore case.” Re-read the problem.
- Removing characters from a Python string with `.replace` inside a loop. `.replace` allocates a new string every call - $O(N^2)$. Build a list and join once.
- Backspace String Compare (LC 844): the obvious stack solution is $O(N)$ space. The two-pointer FROM-THE-END version is $O(1)$ space but tricky - handle counts of '#' carefully.

Warm-up first — build the mechanics

Solve this cold before LC 125. LC 344 is in-place reverse with NO skips - locks the `l / r` swap rhythm before any character filtering enters the picture.

#	Problem	Difficulty	Link
344	Reverse String - in-place two-pointer reverse; the simplest possible two-pointer	Easy	LeetCode

Practice

Solve in this order — each problem unlocks the next.

Step	#	Problem	Difficulty	Link
1	125	Valid Palindrome - the canonical “skip + compare” two-pointer; bar-raiser foundation	Easy	LeetCode
2	151	Reverse Words in a String - split-reverse-join, or the 3-step in-place trick	Medium	LeetCode
3	680	Valid Palindrome II - one deletion allowed; classic two-pointer with one branch	Easy	LeetCode
4	844	Backspace String Compare - two-pointer from the END, count then skip; bar-raiser at Google	Easy	LeetCode

Pattern 2 — Sliding window on a string - “best substring under a constraint”

One-liner: Maintain a window `[l, r]` over the string. Grow `r`; while the window violates the constraint, shrink `l`. Track the best (longest / shortest / count) at each step.

Mental model

Mental model: Almost every “longest / shortest substring with property X” problem is sliding window. The two halves are GROW (always) and SHRINK (only when constraint broken). A counts[] dict tracks the window contents in O(1) per character.

Variable vs fixed window: Variable = grow until you violate, then shrink (LC 3, 76). Fixed = window of fixed size k slides one step at a time (LC 567 - is a permutation present?).

Invariant: After every iteration of r, the window [l, r] is the SHORTEST window ending at r that satisfies / violates the constraint, depending on phrasing.

Cross-reference: This pattern is also fully covered in the Two-Pointers & Hashing sheet. Use that sheet for non-string sliding windows.

Spot signals

- “Longest substring without repeating characters”
- “Minimum window substring containing all of T”
- “Check if s2 contains a permutation of s1” (fixed-size window)
- “Character replacement: longest substring with at most K replacements”
- Any problem where you'd compute a new counter for every substring - the window lets you maintain it incrementally.

Walkthrough - LC 3 Longest Substring Without Repeating Characters

Hash set tracks the window. Shrink l until the duplicate is removed, then add the new char.

Python

```
def lengthOfLongestSubstring(s):
    seen = set()
    l = 0
    best = 0
    for r, ch in enumerate(s):
        while ch in seen:
            seen.remove(s[l])
            l += 1
        seen.add(ch)
        if r - l + 1 > best: best = r - l + 1
    return best
```

Walkthrough - LC 76 Minimum Window Substring

Two counters: need (from t) and window. `formed` counts how many distinct chars in need are satisfied. Grow until formed == |need|, then shrink while still valid; record min.

Python

```

from collections import Counter

def minWindow(s, t):
    if not t: return ''
    need = Counter(t)
    window = {}
    have = 0
    required = len(need)
    best = (float('inf'), 0, 0)
    l = 0
    for r, ch in enumerate(s):
        window[ch] = window.get(ch, 0) + 1
        if ch in need and window[ch] == need[ch]:
            have += 1
        while have == required:
            if r - l + 1 < best[0]:
                best = (r - l + 1, l, r)
            window[s[l]] -= 1
            if s[l] in need and window[s[l]] < need[s[l]]:
                have -= 1
            l += 1
    return '' if best[0] == float('inf') else s[best[1]:best[2]+1]

```

Common traps

- Shrinking the window inside the GROW phase. The shrink only happens when the constraint is violated (or, for min-window, while still valid). Mixing them creates double-decrement bugs.
- Recomputing the counter for every window. The whole point is to maintain it incrementally - +1 on grow, -1 on shrink.
- Min Window Substring: incrementing `have` whenever a char is added. Only increment when window[ch] EXACTLY EQUALS need[ch] - not greater.
- Permutation in String (LC 567): re-creating the counter for every fixed-size window. Use the sliding-counter trick - add the new, remove the old.
- Character Replacement (LC 424): tracking max_freq exactly every step is expensive. Keep a stale max_freq - it only ever decreases when you shrink, and the answer is still correct.

Warm-up first — build the mechanics

Solve this cold before LC 3. LC 1876 is fixed-window of size 3 - the simplest sliding-window framing, with no shrink phase. Locks the grow / track-best rhythm.

#	Problem	Difficulty	Link
---	---------	------------	------

#	Problem	Difficulty	Link
1876	Substrings of Size Three with Distinct Characters - fixed window of 3; the simplest possible sliding-window	Easy	LeetCode

Practice

Solve in this order — each problem unlocks the next.

Step	#	Problem	Difficulty	Link
1	3	Longest Substring Without Repeating Characters - canonical variable-size window; mandatory FAANG	Medium	LeetCode
2	76	Minimum Window Substring - bar-raiser sliding window with two counters; mandatory at Google	Hard	LeetCode
3	567	Permutation in String - fixed-size window with counter equality check	Medium	LeetCode
4	424	Longest Repeating Character Replacement - variable window with the “stale max” trick	Medium	LeetCode

Pattern 3 — Frequency counting / anagrams - Counter as a fingerprint

One-liner: Reduce a string to its character-frequency dict. Two strings are anagrams iff their Counters are equal. Group, compare, and validate in $O(N)$ total.

Mental model

Mental model: Strings made of the same characters in different orders share the same Counter. So `Counter(s) == Counter(t)` is the anagram test in one line. For grouping (LC 49), use `frozenset(Counter(s).items())` - or, faster, `sorted(s)` - as the dict key.

Fixed-alphabet trick: If the input is lowercase-only, replace dict with a 26-length list (`cnt[ord(ch)-ord('a')]` += 1). Faster, cheaper, and equality compares element-by-element.

Invariant: `Counter(s)` is a complete representation of `s` for any anagram-style question; the order is discarded by design.

Cross-reference: The Two-Pointers & Hashing sheet covers hashing in depth; this section focuses on the string-specific applications.

Spot signals

- “Anagram / valid anagram / group anagrams” verbatim
- “First unique character” / “ratio of letters”
- “Check if s1 is an anagram of any substring of s2” (sliding-window + counter)
- “Sort characters by frequency”
- “Isomorphic strings” / “word pattern” (counter-based or bi-directional map)

Walkthrough - LC 49 Group Anagrams

Use the sorted string as a dict key. Map each input string to its sorted form; collect by group.

Python

```
from collections import defaultdict

def groupAnagrams(strs):
    groups = defaultdict(list)
    for s in strs:
        groups[''.join(sorted(s))].append(s)
    return list(groups.values())
```

Walkthrough - LC 242 Valid Anagram (fixed-alphabet)

26-length int array. Bump for s, decrement for t. All zeros at the end → anagram.

Python

```
def isAnagram(s, t):
    if len(s) != len(t): return False
    cnt = [0] * 26
    for a, b in zip(s, t):
        cnt[ord(a) - ord('a')] += 1
        cnt[ord(b) - ord('a')] -= 1
    return all(c == 0 for c in cnt)
```

Common traps

- Forgetting the length check before per-character iteration. Different lengths can't be anagrams - abort early.
- Using sorted strings as dict keys for huge inputs. sort is $O(L \log L)$ per string; the fixed-alphabet tuple/Counter is $O(L)$. Different time profile.
- Group Anagrams: using Counter directly as a key without converting to tuple / frozenset. Counter isn't hashable.
- First Unique Character (LC 387): two-pass is correct (count, then scan). One-pass with OrderedDict is over-engineering for this problem.

- Isomorphic Strings (LC 205): a one-way map is not enough - you need bidirectional. Two dicts ($s \rightarrow t$ and $t \rightarrow s$) or zip + len-of-set comparison.

Warm-up first — build the mechanics

Solve this cold before LC 49. LC 242 is the simplest anagram check - just two Counters compared. Locks the Counter-as-fingerprint mental model.

#	Problem	Difficulty	Link
242	Valid Anagram - single-pair Counter comparison; the simplest possible anagram check	Easy	LeetCode

Practice

Solve in this order — each problem unlocks the next.

Step	#	Problem	Difficulty	Link
1	49	Group Anagrams - the canonical group-by-fingerprint; mandatory FAANG	Medium	LeetCode
2	387	First Unique Character - two-pass counter; the simplest “use a Counter then scan” problem	Easy	LeetCode
3	438	Find All Anagrams in a String - fixed-size sliding window with counter equality; bridge to Pattern 2	Medium	LeetCode
4	205	Isomorphic Strings - bi-directional char map; classic FAANG warm-up	Easy	LeetCode
5	187	Repeated DNA Sequences - hash every fixed-length-10 substring, return those seen twice; the canonical fixed-k fingerprint	Medium	LeetCode

Pattern 4 — Palindromes - expand around centre, Manacher (FYI)

One-liner: For each of the $2N-1$ possible centres, expand outward while characters match. Counts or finds the longest palindromic substring in $O(N^2)$. Manacher's $O(N)$ is FYI - rarely interview-required.

Mental model

Mental model: A palindrome has either an ODD centre (single character) or an EVEN centre (between two characters). $2N-1$ candidates. For each centre, two pointers walk outward; while $s[l] == s[r]$ and in bounds, $l--$ / $r++$. Record best length / count.

Why not the $O(N^3)$ brute force: Substring + isPalindrome for every pair would be $O(N^3)$. Expand-around-centre is $O(N^2)$ by avoiding redundant comparisons.

DP alternative: $dp[i][j] = \text{True}$ iff $s[i..j]$ is a palindrome. $dp[i][j] = (s[i] == s[j]) \text{ AND } dp[i+1][j-1]$. Iterate by length - see Interval DP in the DP sheet.

Manacher's $O(N)$: Uses palindrome symmetry across an established centre to reuse comparisons. Memorise the high-level idea; rarely required to code in an interview.

Spot signals

- “Longest palindromic substring”
- “Count palindromic substrings” (LC 647)
- “Palindrome number” / “Is integer a palindrome” (LC 9 - reverse-half trick)
- “Palindrome partitioning” / “min cuts” (DP variant - see DP sheet)
- “shortest palindrome” / “palindrome pairs” - Manacher's or string-hash territory.

Walkthrough - LC 5 Longest Palindromic Substring (expand around centre)

Try BOTH odd centre (i) and even centre (i, i+1) at each i. Track best length + start.

Python

```
def longestPalindrome(s):
    if not s: return ''
    start = 0
    end = 0
    def expand(l, r):
        while l >= 0 and r < len(s) and s[l] == s[r]:
            l -= 1; r += 1
        return l + 1, r - 1
    for i in range(len(s)):
        for l, r in (expand(i, i), expand(i, i + 1)):
            if r - l > end - start:
                start, end = l, r
    return s[start:end + 1]
```

Walkthrough - LC 9 Palindrome Number (no string conversion)

Reverse the SECOND HALF of the integer in place, then compare to the first half.

Python

```
def isPalindrome(x):
    if x < 0 or (x % 10 == 0 and x != 0): return False
    rev = 0
    while x > rev:
```

```

    rev = rev * 10 + x % 10
    x /= 10
return x == rev or x == rev // 10                # odd-length case

```

Common traps

- Trying only ODD centres. Half of all palindromes have even length - you'll miss them.
- Off-by-one on the return of expand. The last l, r INSIDE the loop are the first MISMATCH, so the true palindrome is s[l+1..r-1].
- LC 9: forgetting that negative numbers are never palindromes (the minus sign isn't a digit).
- Counting palindromic substrings (LC 647): forgetting that expand-around-centre counts ALL palindromes at the centre simultaneously - each successful expansion adds 1.
- Reaching for Manacher's in a Medium-tagged problem. The interviewer won't expect it. Expand-around-centre is the standard answer.

Warm-up first — build the mechanics

Solve this cold before LC 5. LC 9 is palindrome detection on an INTEGER - different domain, same logic. Locks the “compare pairs from the outside in” mental model in a smaller space.

#	Problem	Difficulty	Link
9	Palindrome Number - reverse-half trick; the simplest palindrome problem	Easy	LeetCode

Practice

Solve in this order — each problem unlocks the next.

Step	#	Problem	Difficulty	Link
1	5	Longest Palindromic Substring - expand around centre; mandatory FAANG	Medium	LeetCode
2	647	Palindromic Substrings - count instead of longest; same skeleton	Medium	LeetCode
3	516	Longest Palindromic Subsequence - NOT substring; DP version (interval DP)	Medium	LeetCode
4	131	Palindrome Partitioning — backtracking + palindrome check; see the Backtracking sheet	Medium	LeetCode

Pattern 5 — Pattern matching - substring search (strStr / KMP / Rabin-Karp)

One-liner: Find every occurrence of pattern P in text T. Naive is $O(NM)$. KMP and Rabin-Karp are $O(N+M)$. KMP is the interview standard; Rabin-Karp is the rolling-hash classic.

Mental model

Naive: For each start i in T , compare $P[0..M-1]$ against $T[i..i+M-1]$. Worst case $O(NM)$. Fine for small M .

KMP - longest proper prefix-suffix (LPS) table: $lps[i]$ = length of the longest proper prefix of $P[0..i]$ that is also a suffix. Build LPS in $O(M)$. Then scan T with a pointer j into P ; on mismatch, jump $j = lps[j-1]$ instead of restarting at 0. $O(N+M)$ total.

Rabin-Karp - rolling hash: Hash the current window of T in $O(1)$ per step using a rolling polynomial hash. Compare hash to P 's hash; on match, double-check chars to avoid false positives. Average $O(N+M)$; worst case $O(NM)$.

Built-in (Python): `text.find(pattern)` / `text.index(pattern)` use Two-Way (Crochemore-Perrin) under the hood - $O(N+M)$ average. In real interviews, knowing the API + when to escalate to KMP / Rabin-Karp is the actual signal.

Spot signals

- “Find needle in haystack” / “strStr” / “implementIndexOf”
- “Repeated substring pattern” (LC 459 - `(s+s)[1:-1].contains(s)` trick)
- “Rotated string” - `(s1 + s1).contains(s2)` trick
- “Min appended chars to make a substring” - Z-algorithm or KMP failure function
- Any text-search problem where $N \times M$ would TLE - escalate to KMP / Rabin-Karp.

Walkthrough - LC 28 Find the Index of the First Occurrence (KMP)

Two phases: build the LPS array for the pattern, then scan the text.

Python

```
def strStr(t, p):
    if not p: return 0
    M, N = len(p), len(t)
    # Phase 1 - build LPS
    lps = [0] * M
    length = 0
    i = 1
    while i < M:
        if p[i] == p[length]:
            length += 1
            lps[i] = length
            i += 1
        elif length:
```

```

        length = lps[length - 1]
    else:
        lps[i] = 0
        i += 1
# Phase 2 - scan text
i = j = 0
while i < N:
    if t[i] == p[j]:
        i += 1; j += 1
        if j == M: return i - j
    elif j:
        j = lps[j - 1]
    else:
        i += 1
return -1

```

Walkthrough - LC 459 Repeated Substring Pattern (one-line trick)

A string s is a repetition of a substring iff s appears in $(s + s)[1:-1]$. Two-line proof in the editorial.

Python

```

def repeatedSubstringPattern(s):
    return s in (s + s)[1:-1]

```

Common traps

- KMP LPS construction off-by-one. The base case is $\text{lps}[0] = 0$; the recurrence walks back via $\text{length} = \text{lps}[\text{length} - 1]$ on mismatch, NOT $\text{length} - 1$.
- Confusing pattern index j with text index i . On a match increment BOTH; on a mismatch with $j > 0$ only j moves (via lps).
- Rabin-Karp without modulo. Polynomial hash without mod overflows / collides. Use a big prime (e.g. $10^9 + 7$) and verify on match.
- Reaching for KMP in trivial cases. If $M \leq \sim 20$ and $N \leq \sim 10^5$, naive in Python's C loop is FASTER than your hand-written KMP.
- Forgetting that LC 28 is solvable by `t.find(p)` - don't over-engineer. Mention KMP as the asymptotic answer if asked, code the one-liner first.

Pattern matching has no Easy LC warm-up that exercises KMP itself. Solve LC 28 with naive Python first (return `t.find(p)` or write a nested loop), then re-solve with KMP. The contrast cements WHY KMP exists.

Practice

Solve in this order — each problem unlocks the next.

Step	#	Problem	Difficulty	Link
1	28	Find the Index of the First Occurrence in a String - strStr; KMP territory	Easy	LeetCode
2	459	Repeated Substring Pattern - the (s + s)[1:-1] trick; the most elegant string one-liner	Easy	LeetCode
3	686	Repeated String Match - minimum repeats of a to make it contain b; KMP / find-based	Medium	LeetCode
4	796	Rotate String - (s + s).contains(goal); one-line classic	Easy	LeetCode

Pattern 6 — String parsing - tokenise, validate, convert

One-liner: Walk the string character-by-character with EXPLICIT state. The bug surface is huge - edge cases (empty, leading whitespace, sign, overflow, mixed formats) decide the verdict.

Mental model

Mental model: Parsing problems aren't about an algorithm - they're about ENUMERATING edge cases. Write the state diagram in the comments: "State 0 = skipping whitespace, State 1 = read sign, State 2 = read digits, State 3 = done." Then code it state-by-state. The atoi family is the canonical example.

Token-then-process: For multi-part inputs (version strings, IP addresses, time formats), SPLIT on the delimiter first, then validate each token. Don't try to do both in one pass.

Overflow guards: For atoi / atol, check the answer BEFORE multiplying by 10 again. The classic guard: `if result > (INT_MAX - digit) // 10: return INT_MAX``.

Invariant: After each character, the running result + state EXACTLY represent what's been parsed so far. No look-ahead, no look-behind.

Spot signals

- "String to integer / atoi"
- "Compare version numbers" / "valid IP address" / "valid time / date"
- "Longest common prefix"
- "Roman to integer" / "integer to roman"
- "Decode string" / "basic calculator" - stack-based parsing (see the Stack sheet).

Walkthrough - LC 8 String to Integer (atoi)

State machine: skip whitespace → read optional sign → read digits with overflow check → stop.

Python

```
def myAtoi(s):
    INT_MAX = 2**31 - 1
    INT_MIN = -2**31
    i, n = 0, len(s)
    while i < n and s[i] == ' ': i += 1           # 1. whitespace
    if i == n: return 0
    sign = 1
    if s[i] in ('+', '-'):                         # 2. sign
        sign = -1 if s[i] == '-' else 1
        i += 1
    result = 0
    while i < n and s[i].isdigit():               # 3. digits
        digit = ord(s[i]) - ord('0')
        if result > (INT_MAX - digit) // 10:      # overflow guard
            return INT_MAX if sign == 1 else INT_MIN
        result = result * 10 + digit
        i += 1
    return sign * result
```

Walkthrough - LC 14 Longest Common Prefix

Vertical scan: at column j, every string must have the same char. First mismatch (or end of any string) terminates.

Python

```
def longestCommonPrefix(strs):
    if not strs: return ''
    for j in range(len(strs[0])):
        ch = strs[0][j]
        for s in strs[1:]:
            if j == len(s) or s[j] != ch:
                return strs[0][:j]
    return strs[0]
```

Common traps

- atoi: handling overflow AFTER it happens. Check BEFORE multiplying by 10.
- Version comparison (LC 165): comparing dot-split tokens as strings. "10" < "9" lexicographically. Convert to int FIRST.

- Roman to Integer: forgetting the subtraction pairs (IV, IX, XL, XC, CD, CM). One conditional `if curr < next: subtract`. Don't enumerate the six pairs separately.
- Longest Common Prefix: returning early when one string is empty. The horizontal-scan version handles this naturally; the vertical-scan version needs `if j == len(s)` first.
- Basic Calculator (LC 224 / 227): trying to parse without a stack. Use a stack for parentheses, and operator precedence handling — the Stack sheet has a fuller treatment.

Warm-up first — build the mechanics

Solve this cold before LC 8. LC 14 is parsing without state machines - just compare strings column by column. Locks the “walk indices, watch boundaries” rhythm before atoi piles on edge cases.

#	Problem	Difficulty	Link
14	Longest Common Prefix - vertical-scan classic; warm-up for any string-parsing problem	Easy	LeetCode

Practice

Solve in this order — each problem unlocks the next.

Step	#	Problem	Difficulty	Link
1	8	String to Integer (atoi) - the canonical parsing problem; bar-raiser for edge-case handling	Medium	LeetCode
2	165	Compare Version Numbers - split-on-dot then compare ints; iconic FAANG warmup	Medium	LeetCode
3	13	Roman to Integer - single-pass with subtraction-pair rule; classic	Easy	LeetCode
4	38	Count and Say - iterative run-length encoding; deceptively trap-laden	Medium	LeetCode
5	58	Length of Last Word - walk from the end, skip trailing spaces, count chars; the simplest LC 150 string warm-up	Easy	LeetCode
6	67	Add Binary - schoolbook addition with a carry, right-to-left; the cleanest big-number warm-up	Easy	LeetCode
7	43	Multiply Strings - schoolbook multiplication into a position-indexed buffer; classic big-number bar-raiser	Medium	LeetCode
8	273	Integer to English Words - chunk the number in thousands, spell each chunk; Microsoft staple, hairy edge cases	Hard	LeetCode
9	68	Text Justification - greedy pack words per line, distribute spaces evenly; LC 150 hard, painful edge cases	Hard	LeetCode

Cross-pattern traps (read once before any interview)

Trap	Pattern most at risk	How to dodge
Concatenation inside a loop	1, 5, 6	Build a list, ".join" at the end. Concatenation is $O(N)$ per step $\rightarrow O(N^2)$ total.
Slicing inside a loop ($s = s[1:]$)	1, 6	Use indices. Each slice allocates a new string.
Inner `while l < r` skip without bound check	1	Always `while l < r and (skip-condition)` - NOT just the skip condition.
Recomputing the counter for every window	2, 3	Maintain incrementally: +1 on grow, -1 on shrink.
Only odd-centre palindromes	4	$2N-1$ centres = N odd + $(N-1)$ even. Try BOTH at every i .
KMP without LPS - brute force on big inputs	5	$N*M$ is fine for small inputs; escalate to KMP when $N*M$ would TLE.
atoi: overflow check AFTER multiplying by 10	6	Check BEFORE multiplying. `if result > (INT_MAX - digit) // 10` is the magic line.
Version compare as strings	6	Split on dot, convert tokens to int, then compare. Lex order is wrong ("10" < "9").

Pre-interview revision drill (20 minutes)

Run this the night before:

18. 30 seconds per pattern, name the spot-signal phrase out loud (all 6).
19. 3 minutes per pattern, write the skeleton code from memory (no IDE).
20. 5 minutes, walk through LC 76 (Min Window) - if you can explain why `have == required` is the right loop condition, your sliding window is solid.
21. 5 minutes, walk through LC 5 (Longest Palindromic Substring) - if you can explain why two centre types are needed, your palindrome muscle is solid.
22. 2 minutes, name the #1 trap in each pattern (the cross-pattern table above).

If you fumble any of these $\rightarrow$ don't open more new problems, go re-do the warm-up + canonical problem for that pattern.

7-day study plan

Day	Focus	Problems	Notes
1	Pattern 1 (Two-pointer)	LC 344, 125, 151, 680, 844	Foundation day. By end of day, the l/r loop with skip conditions should flow without thinking.
2	Pattern 2 (Sliding window)	LC 1876, 3, 76, 567, 424	LC 76 deserves a re-solve next morning - it's the bar-raiser version.
3	Pattern 3 (Frequency / anagrams)	LC 242, 49, 387, 438, 205	LC 438 bridges back to Pattern 2 - use it to lock both patterns together.
4	Pattern 4 (Palindromes)	LC 9, 5, 647, 516, 131	Expand-around-centre + palindrome DP + backtracking. Three faces of one topic.
5	Pattern 5 (Pattern matching)	LC 28, 459, 686, 796	Naive first, then KMP. (s+s) trick is the elegant finisher.
6	Pattern 6 (String parsing)	LC 14, 8, 165, 13, 38	atoi (LC 8) is the boss - spend 90 minutes on edge cases.
7	Full revision + retry hardest 3	Re-solve LC 76, 5, 8 cold	If any of these still feels uncertain, re-read the relevant pattern + redo its warm-up.

Total: ~25 problems across 7 days. If a problem takes > 30 minutes without progress, peek at the editorial, understand the trick, write the code from scratch the next day. NEVER copy-paste; the muscle is in the typing.

The big idea

String looks like a separate topic. It isn't - it's 6 array patterns with the alphabet locked to characters. The students who clear FAANG don't memorise 50 string tricks - they own 6 templates and reach for the right one in 10 seconds.

Do the 25 must-do problems. Internalise the patterns. Then every future string problem is just a remix.

Pseudocode-first reading: every code block is shown in Python — read each as pseudocode and translate to your interview language (Java / C++ / TS / Go). Single-language keeps each sheet lean; the patterns travel.